

AdaCtrl: Towards Adaptive and Controllable Reasoning via Difficulty-Aware Budgeting

Shijue Huang^{α*}, Hongru Wang^{β*}, Wanjun Zhong^{*}, Zhaochen Su^α, Jiazhan Feng^σ,
Bowen Cao^β, Yi R. (May) Fung^{α†}

^αHong Kong University of Science and Technology

^βThe Chinese University of Hong Kong, ^σPeking University

Abstract

Modern large reasoning models demonstrate impressive problem-solving capabilities by employing sophisticated reasoning strategies. However, they often struggle to balance efficiency and effectiveness, frequently generating unnecessarily lengthy reasoning chains for simple problems. In this work, we propose AdaCtrl, a novel framework to support both difficulty-aware adaptive reasoning budget allocation and explicit user control over reasoning depth. AdaCtrl dynamically adjusts its reasoning length based on self-assessed problem difficulty, while also allowing users to manually control the budget to prioritize either efficiency or effectiveness. This is achieved through a two-stage training pipeline: an initial cold-start fine-tuning phase to instill the ability to self-aware difficulty and adjust reasoning budget, followed by a difficulty-aware reinforcement learning (RL) stage that refines the model’s adaptive reasoning strategies and calibrates its difficulty assessments based on its evolving capabilities during online training. To enable intuitive user interaction, we design explicit length-triggered tags that function as a natural interface for budget control. Empirical results show that AdaCtrl adapts reasoning length based on estimated difficulty, compared to the standard training baseline that also incorporates fine-tuning and RL, it yields performance improvements and simultaneously reduces response length by 10.06% and 12.14% on the more challenging AIME2024 and AIME2025 datasets, which require elaborate reasoning, and by 62.05% and 91.04% on the MATH500 and GSM8K datasets, where more concise responses are sufficient. Furthermore, AdaCtrl enables precise user control over the reasoning budget, allowing for tailored responses to meet specific needs. Further analysis also reveals that AdaCtrl accurately estimates problem difficulty and allocates reasoning budgets in alignment with these assessments².

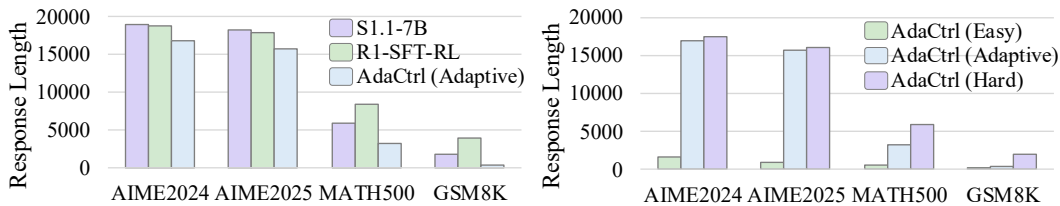


Figure 1: (Left) With enhanced performance (see Table 1), AdaCtrl adaptively adjusts its reasoning length, resulting in better efficiency compared to baselines; (Right) The controllable easy and hard modes of AdaCtrl allow accurate user control of reasoning budgets, as detailed in Table 2.

*Equal contribution. † Corresponding author.

²This work is ongoing, and the code will be released at <https://github.com/JoeYing1019/AdaCtrl1>.

1 Introduction

With the emergence of test-time scaling [Snell et al., 2024, Muennighoff et al., 2025, Balachandran et al., 2025, Zhang et al., 2025], large reasoning models such as Deepseek R1 [DeepSeek-AI et al., 2025] and OpenAI O1 [OpenAI, 2024] have demonstrated superior performance across a variety of tasks by adopting more elaborate thinking strategies prior to generating final answers. While extended reasoning processes can significantly enhance a model’s ability to solve complex problems, they also introduce substantial inference overhead and may lead to the overthinking problem [Chen et al., 2025, Sui et al., 2025]. For instance, even when presented with a simple problem like “Evaluate $\log_2(64)$ ”, these models still tend to engage in lengthy chain-of-thought, exploring multiple solution strategies and employing advanced System-2 thinking strategies [Ryan et al., 2016, Li et al., 2025] such as planning, reflection, and verification, as the response demonstrated in right part of Figure 2.

Although such thorough reasoning is beneficial for complex queries, it results in excessive latency and resource consumption for simpler tasks, negatively impacting user experience.

To address this, recent researches have explored ways to improve reasoning efficiency. Some studies [Nayab et al., 2025, Muennighoff et al., 2025, Xu et al., 2025] leverage prompting techniques that require models to generate concise responses via explicit length constraints. Others [Munkhbat et al., 2025, Ma et al., 2025b, Xia et al., 2025b] fine-tune models on shorter reasoning trajectories. Additionally, with the growing application of reinforcement learning (RL) in LLM training, several approaches [Arora and Zanette, 2025, Aggarwal and Welleck, 2025, Hou et al., 2025] have emerged that use RL with length reward design to penalize overly long responses, thereby optimizing the model to generate outputs that are both concise and accurate. However, most existing work primarily focuses on minimizing the length of reasoning, overlooking the need for adaptive budget mechanisms that dynamically determine reasoning depth to achieve a more cost-effective balance between performance and computational efficiency. Furthermore, from the user’s perspective, a notable advantage is the controllability of the reasoning budget, which allows users to explicitly manage computational resources to prioritize either efficiency or effectiveness as needed. Despite its potential, this aspect remains relatively under-explored in current research.

To establish more adaptive and controllable reasoning approach, we propose AdaCtrl, a novel framework designed to support both adaptive and controllable reasoning. As shown in Figure 2, AdaCtrl allows the model to autonomously balance inference-time computation based on its self-assessed difficulty of the problem (i.e., adaptive reasoning mode), while also enabling users to influence the reasoning budget through designed explicit length-trigger tags (i.e. the specified easy or hard reasoning modes). Specifically, we aim to equip the model with two key meta-capabilities: (1) Self-difficulty awareness, which allows the model to estimate the difficulty of a given problem relative to its own capabilities; (2) Difficulty-aware reasoning budget allocation, which adjusts the length of the model’s response based on either the model’s own assessment or user-specified difficulty levels. To facilitate user-friendly controllable reasoning manner, we first design two length-triggered tags, namely “[Easy]” and “[Hard]”, to serve as natural interface for budget control. Then we introduce a two-stage training framework to realize these capabilities. The first stage involves cold-start fine-tuning to provide the model with an initial ability to use length-trigger tags and allocate reasoning budgets based on difficulty. The second stage employs a difficulty-aware reinforcement learning framework to refine the model’s adaptive reasoning strategies and to calibrate its self-assessment of problem difficulty according to model’s evolving capabilities during online training.

Experimental results across four datasets demonstrate that AdaCtrl effectively enables models to adaptively allocate reasoning budgets through self-assessment of problem difficulty. Compared to the standard SFT + rule-based RL baseline, AdaCtrl achieves comparable accuracy on AIME2024 and yields performance improvements of 1.67% on AIME2025, 7.20% on MATH500, and 2.05% on GSM8K. Notably, these gains are accompanied by substantial reductions in response length by 10.06%, 12.14%, 62.05%, and 91.04% on the respective datasets. Moreover, AdaCtrl offers enhanced controllability via explicit length-trigger tags. When instructed to operate in the easy reasoning mode, the model reduces response lengths by 90.22% and 94.31% (from 16k to 1k tokens) on the AIME2025 and AIME2024 datasets, respectively. Conversely, under the hard reasoning mode, response lengths increase by 86.51% on GSM8K and 489.15% on MATH500. These results underscore AdaCtrl’s capability for dynamic and interpretable control over reasoning depth. Further analysis reveals that AdaCtrl serves as an effective difficulty estimator, and accurately enables difficulty-aware budget allocation. The main contributions of this work are summarized as follows:

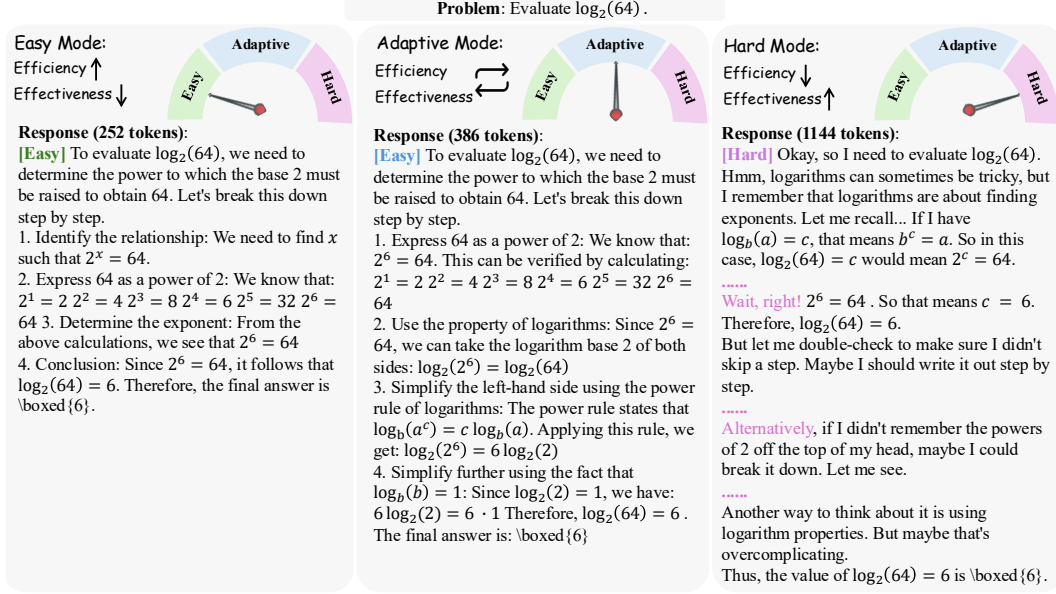


Figure 2: Given same problem, AdaCtrl supports three reasoning modes, the adaptive mode dynamically allocates budgets according to the problem complexity, the easy mode offers concise answers with minimal budgets, and the hard mode delivers extensive responses with larger budgets.

- We propose AdaCtrl, which by facilitating dynamic trade-offs between efficiency and effectiveness, allows the model to emulate human cognitive behavior by modulating reasoning effort in response to varying problem demands and enhances user control on computational resources.
- We introduce a two-stage training paradigm that integrates cold-start fine-tuning and difficulty-aware reinforcement learning. This approach fosters self-awareness of task difficulty and supports difficulty-sensitive allocation of computational resources.
- Empirical results on four benchmark datasets demonstrate that AdaCtrl successfully balances efficiency and effectiveness by adjusting to its self-assessed task difficulty. Furthermore, it offers a flexible interface that allows users to explicitly control the reasoning budget as needed.

2 Related Work

2.1 Reasoning Efficiency via Supervised Fine-Tuning

While LLMs achieve impressive performance on complex tasks by generating elaborate multi-step reasoning chains [Dubey et al., 2024, Su et al., 2025], this capability can lead to excessive verbosity and computational overhead for simpler queries. This “overthinking” phenomenon has motivated research into improving reasoning efficiency [Qu et al., 2025a, Wang et al., 2025]. One prominent strategy involves Supervised Fine-Tuning (SFT) to guide models towards more concise reasoning. Some SFT works focus on training with inherently shorter reasoning paths. For example models learn adherence to token budgets through specific prompting during data generation [Han et al., 2024]. Others distill concise paths from best-of-N sampling [Munkhbat et al., 2025] or fine-tune models to omit intermediate steps for samples where the model is already confident [Yu et al., 2024]. Another SFT direction compresses existing reasoning chains. Kang et al. [2024] employ GPT-4 [Achiam et al., 2023] as a compressor then fine-tune a model on these long-to-short CoT mappings. LMSkip [Liu et al., 2024] induces step-skipping behavior under step constraints. SPIRIT-FT [Cui et al., 2025] identifies critical reasoning steps using perplexity as a guide for pruning. TokenSkip [Xia et al., 2025a] analyzes token importance within CoT outputs for controllable compression. These SFT methods reduce length but often enforce a general conciseness ill-suited for complex problems and typically lack self-assessment of difficulty or user budget control. While CoT-Valve [Ma et al., 2025a] offers some length modulation, our approach uniquely integrates explicit training for self-difficulty awareness and user-facing control tags within a two-stage SFT-RL framework.

2.2 Reasoning Efficiency via Reinforcement Learning

RL offers another significant avenue for optimizing reasoning efficiency, building upon its success in developing deep reasoning capabilities in models like DeepSeek-Coder [Guo et al., 2025]. Many RL approaches incorporate explicit length-based rewards to encourage conciseness alongside accuracy. Some methods link generation length to task difficulty or directives within the prompt: DAST [Shen et al., 2025] adapts CoT length to problem complexity via reward shaping, while LCPO [Aggarwal and Welleck, 2025] controls length using prompt-specified targets. Other techniques normalize length rewards against baselines, as seen in O1-Pruner [Luo et al., 2025], the per-prompt normalization by Arora and Zanette [2025], and the Kimi 1.5 report [Team et al., 2025]. Yeo et al. [2025] proposed a cosine reward to manage length effectively, also highlighting the “length hacking” problem where models artificially extend reasoning. Beyond explicit length rewards, alternative RL strategies include meta-RL for test-time optimization [Qu et al., 2025b], utility maximization for budget awareness [Yu et al., 2025b], preference optimization with heuristics [Chen et al., 2025], and mitigating GRPO’s bias towards longer trajectories [Liu et al., 2025]. However, these methods generally lack the explicit user control over reasoning depth offered by AdaCtrl’s length-trigger tags and do not prioritize training for self-awareness of problem difficulty. Our two-stage SFT-RL framework uniquely addresses these aspects, enabling both autonomous and user-influenced reasoning budgets.

3 Method

In this section, we demonstrate the design of our proposed AdaCtrl, which includes: (1) Cold-start fine-tuning that provides effective initialization for output format and difficulty estimation; (2) Difficulty-aware reinforcement learning framework that boosts model’s behavior on response length control and difficulty estimation. The whole framework is demonstrated in Figure 3.

3.1 Length-Trigger Tags as Controlling Interface

As illustrated in Figure 3, we introduce two special length-trigger tags, “[Easy]” and “[Hard]”, to more effectively utilize problem difficulty as a signal for controlling response length. Each model-generated response begins with one of these tags, indicating the anticipated complexity of the required reasoning. When presented with a problem, the model is instructed to generate a response prefaced by one of the length-trigger tags, thereby conditioning its subsequent reasoning process on the chosen tag. By equipping the model with the ability to allocate its reasoning resources according to the specified tag, we provide a straightforward interface for controlling response length: users simply supply the preferred length-trigger tag as an initial input token.

3.2 Cold-Start Fine-Tuning

This stage primarily focuses on equipping models with the ability to adhere to output formats that include length-triggered tags and to control response length accordingly. To curate suitable training data for this purpose, we directly select both easy and hard problems from the DeepMATH dataset [He et al., 2025], which provides difficulty annotations for each problem. For easy problems $\{q_1^e, q_2^e, \dots, q_n^e\}$, we utilize the model to be trained $\mathcal{M}$ (i.e. Qwen2.5-7B-Instruct) to generate concise response, while for hard ones $\{q_1^h, q_2^h, \dots, q_m^h\}$, a strong large reasoning model $\mathcal{R}$ is employed to generate reasoning trace. Then we filter the correct part for both subset:

$$\mathcal{D}^e = \{(q_i^e, [r_i, y_i]) | [r_i, y_i] = \mathcal{M}(q_i^e), \mathbb{I}(y_i, \hat{y}_i) = 1\}, \quad (1)$$

$$\mathcal{D}^h = \{(q_i^h, [r_i, y_i]) | [r_i, y_i] = \mathcal{R}(q_i^h), \mathbb{I}(y_i, \hat{y}_i) = 1\}, \quad (2)$$

where $[r_i, y_i]$ is the model-generated response that includes reasoning process r_i and the predicted answer y_i , $\hat{y}_i$ is the ground truth answer of corresponding samples, and $\mathbb{I}(y_i, \hat{y}_i) = 1$ represents that the model response $[r_i, y_i]$ is correct with the verification of answer $\hat{y}_i$.

By prepending the “[Easy]” tag for response in $\mathcal{D}^e$ and “[Hard]” for those in $\mathcal{D}^h$, we construct a dataset $\mathcal{D} = \{(q_i, [t_i, r_i, y_i])\}$ for supervised fine-tuning (SFT) as cold start, where t_i is length-trigger tag. After the cold start finetuning, model generated responses will start with above length-trigger tags.

when the generated length-trigger tag t_i is “[Easy]”:

$$r_l(y_i) = \begin{cases} 1.0 - \frac{1 - \cos((l_i^j / L_i)\pi)}{2} & t_i = [\text{Easy}] \\ 0.0 & \text{otherwise} \end{cases} \quad (5)$$

where l_i^j is the j -th rollout length for problem q_i , and L_i is the max length in the rollout group of problem q_i . Utilizing the monotonicity of cosine function within a specific domain (i.e. $0 - \pi$), the difficulty-aware length reward will assign lower score with the increase of generated response length.

3.3.2 Reinforcement Learning for Difficulty-Aware Budgeting

To enhance the model’s adaptive reasoning capabilities and calibrate its self-evaluation of problem difficulty during online training, the overall reward in the reinforcement learning process is computed by integrating three specifically designed reward components:

$$r(y_i) = r_o(y_i) + \alpha \cdot r_f(y_i) + \beta \cdot r_l(y_i) \quad (6)$$

where α and β are hyper-parameters.

At this stage, we sampled another collection of problems that covers both easy and hard problems from DeepMATH [He et al., 2025] to allow the model to learn dynamic strategies for different types of problems. Following Yu et al. [2025a], He et al. [2025], the policy model π_θ is optimized through the following objective:

$$\begin{aligned} \mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{(q,a) \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|q)} & \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left(\min \left(\frac{\pi_\theta(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} | q, o_{i,<t})} \right) (\theta) \hat{A}_{i,t}, \right. \right. \\ & \left. \left. \text{clip} \left(\frac{\pi_\theta(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} | q, o_{i,<t})} (\theta), 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right) - \beta D_{\text{KL}}(\pi_\theta || \pi_{\text{ref}}) \right] \end{aligned} \quad (7)$$

where G is the group size, o_i is the rollout response, $\hat{A}_{i,t}$ is the advantage of the i -th response calculated by normalizing rewards in the group, and β, ε are hyper-parameters.

Through reinforcement learning, models can more effectively assess problem difficulty relative to their own capabilities and develop adaptive reasoning strategies accordingly.

4 Experiment

4.1 Experimental Setup

Model and Datasets. We adopt Qwen2.5-7B-Instruct [Qwen et al., 2025] as the backbone model for training. During the cold-start fine-tuning phase, the training problems are drawn from the DeepMATH dataset [He et al., 2025], which assigns each problem a difficulty level ranging from 1 to 9. We categorize problems with difficulty levels of 5 or below as easy, and those above 5 as hard. For easy problems, Qwen2.5-7B-Instruct is employed to generate concise responses. In contrast, for hard problems, we incorporate extensive long-form reasoning trajectories generated by Deepseek R1 [DeepSeek-AI et al., 2025]. After filtering these trajectories using ground-truth answers, we construct a fine-tuning dataset comprising 8K instances, which includes 4K with short and 4K with long reasoning chains. To support difficulty-aware reinforcement learning, we further sample an additional 30K examples from DeepMATH dataset that comprise 10K easy and 20K hard problems and are distinct from those in the cold-start fine-tuning dataset.

Training Details. For cold-start fine-tuning, we employ the ms-swift framework [Zhao et al., 2024] for training, using a learning rate of $1e-5$ and a batch size of 8. For difficulty-aware reinforcement learning, we adopt VeRL [Sheng et al., 2025], with all reinforcement learning experiments conducted under a unified setting. Specifically, we follow DAPO [Yu et al., 2025a] to eliminate KL divergence. The policy model is optimized using the AdamW optimizer with a learning rate of $1e-6$, a batch size of 256, and a micro-batch size of 32. And we set the value of α and β both as 0.5 during training. During the rollout phase, 16 responses are sampled per prompt, and the maximum generation length is set to 24K tokens. All experiments are conducted on NVIDIA H800 GPUs.

Evaluation Settings. We evaluate our method on four mathematical datasets that span both easy and challenging problems: AIME2024 [Art of Problem Solving, n.d.], AIME2025 [Art of Problem Solving, n.d.], MATH500 [Hendrycks et al., 2021], and GSM8K [Cobbe et al., 2021]. The first two datasets consist of more challenging Math Olympiad-style problems, whereas the latter two primarily contain simpler, grade-school level problems, with GSM8K being the easiest among them. Since AIME2024 and AIME2025 each contain only 30 samples, we report the average performance over 8 independent runs for these two datasets. All evaluations are conducted using consistent inference hyper-parameters set to a temperature of 0.7 and a top-p value of 0.8. To quantify the effectiveness and efficiency of models, we report two metrics: accuracy (Acc.) and the number of tokens generated in the response (Len.).

Baselines. To assess the effectiveness of AdaCtrl, we compare it against several baseline methods that share the same backbone architecture. These include: (1) Qwen2.5-7B-Instruct: the unmodified base instruction-tuned model; (2) S1.1-7B: a reasoning-enhanced model obtained by further training Qwen2.5-7B-Instruct on the S1.1.1K dataset [Muennighoff et al., 2025]; (3) R1-SFT: a baseline model fine-tuned Qwen2.5-7B-Instruct on a curated dataset provided by Deepseek R1, which includes problems sourced from the cold-start SFT dataset along with all responses generated by Deepseek R1; (4) R1-SFT-RL: a reinforcement learning variant of (3), trained with outcome accuracy rewards on the aforementioned 30K RL dataset; (5) Cold-Start: a model fine-tuned Qwen2.5-7B-Instruct on the constructed cold-start SFT dataset; (6) Cold-Start-RL: the reinforcement learning counterpart of (5), trained using outcome accuracy rewards on the same 30K RL dataset.

Table 1: Main results. We compute two metrics: Acc. is average accuracy and Len. is the average generated tokens of response.

Model	AIME2024		AIME2025		MATH500		GSM8K	
	Acc.	Len.	Acc.	Len.	Acc.	Len.	Acc.	Len.
Qwen2.5-7B-Instruct	11.25	1805.60	7.08	1174.06	73.00	628.91	91.58	272.93
S1.1-7B	16.67	19022.60	18.33	18302.72	68.80	5824.47	91.89	1838.12
R1-SFT	12.08	20767.36	13.33	20184.36	62.00	8103.41	87.34	3336.81
R1-SFT-RL	21.25	18778.92	17.50	17924.35	66.80	8421.57	88.93	3896.76
Cold-Start	11.25	5553.00	7.92	6237.49	71.00	805.78	90.22	345.44
Cold-Start-RL	18.33	16911.63	14.58	15941.76	73.00	3797.60	90.67	369.94
AdaCtrl (Adaptive)	21.25	16889.50	19.17	15749.08	74.00	3195.69(▼ 62.05%)	90.98	349.34(▼ 91.04%)

4.2 Main Results

The main results are presented in Table 1. We can have the following observations:

AdaCtrl effectively balances performance and reasoning budget. Compared to the RL baseline R1-SFT-RL, AdaCtrl achieves comparable accuracy on AIME2024, and improves performance by 1.67% on AIME2025, 7.20% on MATH500, and 2.05% on GSM8K. Simultaneously, it substantially reduces response lengths by 10.06%, 12.14%, 62.05%, and 91.04% on these datasets, respectively. When compared to S1.1-7B, AdaCtrl further demonstrates gains of 4.58%, 0.84%, and 5.20% on AIME2024, AIME2025, and MATH500, respectively, and achieves competitive results on GSM8K. Corresponding reductions in response length for this comparison are 11.21%, 13.95%, 45.13%, and 80.99%. These results suggest that AdaCtrl is capable of adaptively allocating reasoning budget based on problem difficulty, thereby achieving an effective trade-off between reasoning efficiency and effectiveness.

Cold-start fine-tuning offers a strong initialization for adaptive budgeting. In contrast to R1-SFT, which is trained exclusively on long-form reasoning traces from Deepseek R1, our cold-start fine-tuning approach leverages a mixture of concise and extended reasoning trajectories. This enables the model to develop more efficient reasoning strategies, leading to substantial reductions in response length by 73.26%, 69.10%, 90.06%, and 89.65% on AIME2024, AIME2025, MATH500, and GSM8K, respectively. Moreover, when further trained with rule-based RL, models initialized from cold-start checkpoint (Cold-Start-RL) demonstrate more accurate budget control compared with that initialized from R1-SFT (i.e., R1-SFT-RL), reducing response lengths by 9.94%, 11.06%, 54.91%, and 91.04% across the same datasets. These findings highlight the importance of the cold-start fine-tuning stage in providing a solid foundation for subsequent adaptive budgeting.

Designed reward functions enhance reasoning effectiveness. Using the cold-start fine-tuning model as backbone, AdaCtrl surpasses Cold-Start-RL that employs reinforcement learning based solely on outcome accuracy, achieving accuracy gains of 2.92%, 4.59%, 1.00%, and 0.31% on AIME2024, AIME2025, MATH500, and GSM8K, respectively. These improvements can be attributed to our reward design, which incorporates both difficulty estimation calibration and difficulty-aware length adjustments. These additional reward signals promote more accurate assessments of problem complexity and allocation of computational resources, leading to a more nuanced balance between effective and efficiency.

4.3 Analysis

Table 2: The comparison between adaptive and controlled reasoning modes of AdaCtrl.

AdaCtrl	AIME2024		AIME2025		MATH500		GSM8K	
	Acc.	Len.	Acc.	Len.	Acc.	Len.	Acc.	Len.
Adaptive	21.25	16889.50	19.17	15749.08	74.00	3195.69	90.98	349.34
Easy	14.58	1652.42(▼ 90.22%)	10.00	896.14(▼ 94.31%)	70.80	652.76	90.75	314.49
Hard	21.67	17562.94	22.08	16114.87	71.20	5960.15	92.57	2058.13(▲ 489.15%)

4.3.1 Controllability of AdaCtrl

AdaCtrl introduces length-trigger tags that can be used as prerequisite tokens during generation to provide explicit control over response length. To evaluate the controllability of this mechanism, we simulate user intent by specifying either the “[Easy]” or “[Hard]” tag, corresponding to simplified and complex reasoning modes, respectively. The experimental results, summarized in Table 2, reveal clear trends. Compared to the adaptive reasoning mode, where the model autonomously infers the problem’s difficulty, forcing the “[Easy]” mode leads to a consistent drop in performance across all four datasets. However, it also results in a substantial reduction in response length, especially by 90.22% and 94.31% on the more challenging AIME2024 and AIME2025 datasets. In contrast, the “[Hard]” mode improves performance on most datasets and significantly increases response length, with gains of 86.51% and 489.15% on the simpler MATH500 and GSM8K datasets. These results indicate that AdaCtrl provides accurate user-level control over reasoning budgets. Furthermore, the adaptive reasoning mode demonstrates a favorable balance between effectiveness and efficiency. This suggests that our proposed framework not only enables fine-grained control but also offers enhanced flexibility to accommodate diverse requirements in practical deployments.

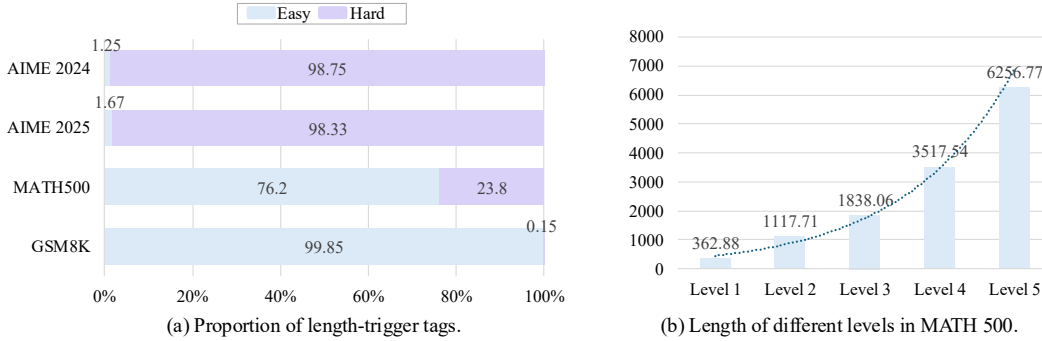


Figure 4: (a) The proportion of length-trigger tags across different datasets; and (b) the length of response in different levels of problems in MATH500.

4.3.2 AdaCtrl Serves as Good Difficulty Estimator

To assess the problem difficulty estimation capability of AdaCtrl, we analyze the proportion of length-triggered tags generated by the model across four datasets, which serve as indicators of the model’s difficulty judgments. As shown in Figure 4 (a), AdaCtrl classifies the majority of problems in AIME2024 and AIME2025 datasets to the hard category, which include challenging math olympiad level problems. In contrast, for the MATH500 dataset, which contains a mix of easy and difficult

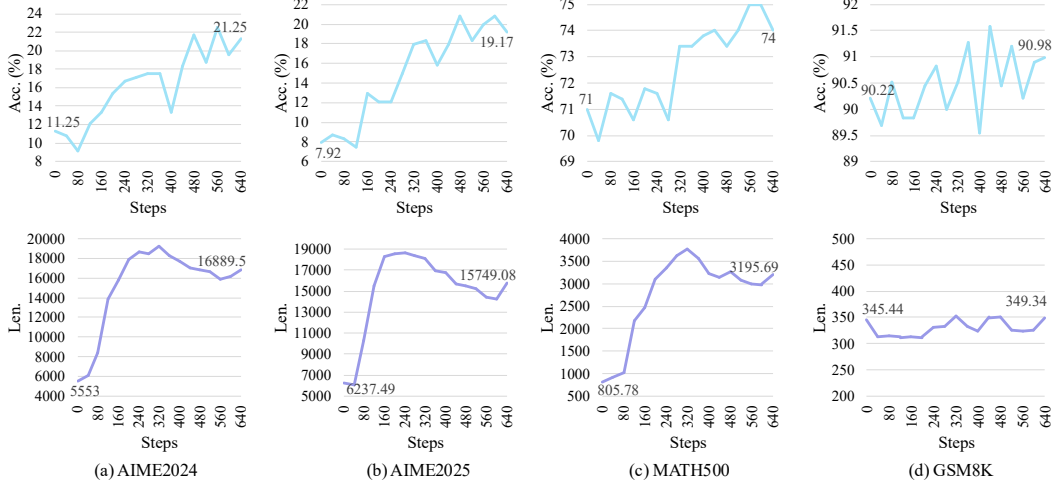


Figure 5: Training dynamics during RL.

problems (most of which are relatively solvable by current large language models), the model identifies 76.2% of the problems as easy ones. For GSM8K, the simplest dataset among the four, over 99% of the problems are categorized as easy ones. These results are consistent with the actual difficulty levels of the datasets and demonstrate that AdaCtrl develops a satisfactory capability to estimate problem difficulty through reinforcement learning.

4.3.3 AdaCtrl Facilitates Accurately Difficulty-Aware Budgeting

To further investigate the adaptive difficulty-aware budgeting capabilities of AdaCtrl, we analyze its responses on the MATH500 dataset, which provides difficulty level annotations for each problem. As illustrated in Figure 4 (b), AdaCtrl generates progressively longer and more elaborate responses as the difficulty level increases from 1 to 5, ranging from approximately 0.3K to 6K tokens. This trend indicates that AdaCtrl can accurately regulate its reasoning budget based on its self-assessed estimation of problem difficulty, thereby enabling automatic and adaptive allocation of computational resources.

4.3.4 Training Dynamics During RL

To further investigate how the model learns to allocate adaptive reasoning budgets, we analyze both performance trends and budget dynamics throughout the reinforcement learning (RL) training process. As illustrated in Figure 5, the model exhibits an upward trends in performance across all four datasets, suggesting a progressive enhancement in reasoning capabilities. Regarding budget dynamics, we observe distinct patterns across datasets. On AIME2024, AIME2025, and MATH500, the average response length initially increases rapidly during early training steps, then gradually decreases and stabilizes at a level longer than the that before RL training. While for GSM8K, the response length remains relatively stable and close to that observed before RL training.

These findings suggest that the reasoning budget allocation learned during cold-start fine-tuning is insufficient for more complex problems, such as those in AIME2024, AIME2025, and MATH500. Consequently, the model adjusts its budget dynamically in response to actual problem difficulty during RL phase. In contrast, for the comparatively simpler GSM8K dataset, the model is already capable of effectively allocating minimal budgets after cold-start fine-tuning, indicating its ability to distinguish and handle easier problems without requiring significant adjustment.

5 Conclusion

In this work, we propose an adaptive and controllable reasoning framework designed to mitigate the problem of overthinking while granting users explicit control over computational resources. To this end, we introduce AdaCtrl, a model that supports both dynamic reasoning budget allocation

and user-directed budget adjustments. Our approach utilizes a two-stage training pipeline that combines cold-start fine-tuning with difficulty-aware reinforcement learning. Experiments conducted on four benchmark datasets demonstrate that AdaCtrl effectively allocates reasoning budgets based on self-assessed problem difficulty, leading to performance improvements while dynamically reducing response lengths by 10%–90%. This enables flexible trade-offs between efficiency and performance. Furthermore, AdaCtrl provides an intuitive user interface that allows users to directly adjust reasoning budgets according to their need.

References

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Pranjal Aggarwal and Sean Welleck. L1: Controlling how long a reasoning model thinks with reinforcement learning, 2025. URL <https://arxiv.org/abs/2503.04697>.
- Daman Arora and Andrea Zanette. Training language models to reason efficiently, 2025. URL <https://arxiv.org/abs/2502.04463>.
- Art of Problem Solving. Aime problems and solutions. https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions, n.d. Accessed: 2025-05-22.
- Vidhisha Balachandran, Jingya Chen, Lingjiao Chen, Shivam Garg, Neel Joshi, Yash Lara, John Langford, Besmira Nushi, Vibhav Vineet, Yue Wu, and Safoora Yousefi. Inference-time scaling for complex tasks: Where we stand and what lies ahead, 2025. URL <https://arxiv.org/abs/2504.00294>.
- Xingyu Chen, Jiahao Xu, Tian Liang, Zhiwei He, Jianhui Pang, Dian Yu, Linfeng Song, Qiuzhi Liu, Mengfei Zhou, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. Do not think that much for $2+3=?$ on the overthinking of o1-like llms, 2025. URL <https://arxiv.org/abs/2412.21187>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- Yingqian Cui, Pengfei He, Jingying Zeng, Hui Liu, Xianfeng Tang, Zhenwei Dai, Yan Han, Chen Luo, Jing Huang, Zhen Li, et al. Stepwise perplexity-guided refinement for efficient chain-of-thought reasoning in large language models. *arXiv preprint arXiv:2502.13260*, 2025.
- DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyi Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojuan Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanjia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Chen, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li,

- Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Tingxu Han, Chunrong Fang, Shiyu Zhao, Shiqing Ma, Zhenyu Chen, and Zhenting Wang. Token-budget-aware llm reasoning. *arXiv preprint arXiv:2412.18547*, 2024.
- Zhiwei He, Tian Liang, Jiahao Xu, Qiuzhi Liu, Xingyu Chen, Yue Wang, Linfeng Song, Dian Yu, Zhenwen Liang, Wenxuan Wang, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. Deepmath-103k: A large-scale, challenging, decontaminated, and verifiable mathematical dataset for advancing reasoning, 2025. URL <https://arxiv.org/abs/2504.11456>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset, 2021. URL <https://arxiv.org/abs/2103.03874>.
- Bairu Hou, Yang Zhang, Jiabao Ji, Yujian Liu, Kaizhi Qian, Jacob Andreas, and Shiyu Chang. Thinkprune: Pruning long chain-of-thought of llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2504.01296>.
- Yu Kang, Xianghui Sun, Liangyu Chen, and Wei Zou. C3ot: Generating shorter chain-of-thought without compromising effectiveness. *arXiv preprint arXiv:2412.11664*, 2024.
- Zhong-Zhi Li, Duzhen Zhang, Ming-Liang Zhang, Jiaxin Zhang, Zengyan Liu, Yuxuan Yao, Haotian Xu, Junhao Zheng, Pei-Jie Wang, Xiuyi Chen, Yingying Zhang, Fei Yin, Jiahua Dong, Zhiwei Li, Bao-Long Bi, Ling-Rui Mei, Junfeng Fang, Zhijiang Guo, Le Song, and Cheng-Lin Liu. From system 1 to system 2: A survey of reasoning large language models, 2025. URL <https://arxiv.org/abs/2502.17419>.
- Tengxiao Liu, Qipeng Guo, Xiangkun Hu, Cheng Jiayang, Yue Zhang, Xipeng Qiu, and Zheng Zhang. Can language models learn to skip steps? *arXiv preprint arXiv:2411.01855*, 2024.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective. <https://github.com/sail-sg/understand-r1-zero>, 2025.
- Haotian Luo, Li Shen, Haiying He, Yibo Wang, Shiwei Liu, Wei Li, Naiqiang Tan, Xiaochun Cao, and Dacheng Tao. O1-pruner: Length-harmonizing fine-tuning for o1-like reasoning pruning. *arXiv preprint arXiv:2501.12570*, 2025.
- Xinyin Ma, Guangnian Wan, Runpeng Yu, Gongfan Fang, and Xinchao Wang. Cot-valve: Length-compressible chain-of-thought tuning. *arXiv preprint arXiv:2502.09601*, 2025a.
- Xinyin Ma, Guangnian Wan, Runpeng Yu, Gongfan Fang, and Xinchao Wang. Cot-valve: Length-compressible chain-of-thought tuning, 2025b. URL <https://arxiv.org/abs/2502.09601>.
- Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. sl: Simple test-time scaling, 2025. URL <https://arxiv.org/abs/2501.19393>.

- Tergel Munkhbat, Namgyu Ho, Seohyun Kim, Yongjin Yang, Yujin Kim, and Se-Young Yun. Self-training elicits concise reasoning in large language models, 2025. URL <https://arxiv.org/abs/2502.20122>.
- Sania Nayab, Giulio Rossolini, Marco Simoni, Andrea Saracino, Giorgio Buttazzo, Nicolamaria Manes, and Fabrizio Giacomelli. Concise thoughts: Impact of output length on llm reasoning and cost, 2025. URL <https://arxiv.org/abs/2407.19825>.
- OpenAI. Introducing openai o1, September 2024. URL <https://openai.com/o1/>. Accessed: 2025-05-21.
- Xiaoye Qu, Yafu Li, Zhaochen Su, Weigao Sun, Jianhao Yan, Dongrui Liu, Ganqu Cui, Daizong Liu, Shuxian Liang, Junxian He, et al. A survey of efficient reasoning for large reasoning models: Language, multimodality, and beyond. *arXiv preprint arXiv:2503.21614*, 2025a.
- Yuxiao Qu, Matthew Y. R. Yang, Amrith Setlur, Lewis Tunstall, Edward Emanuel Beeching, Ruslan Salakhutdinov, and Aviral Kumar. Optimizing test-time compute via meta reinforcement fine-tuning, 2025b. URL <https://arxiv.org/abs/2503.07572>.
- Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025. URL <https://arxiv.org/abs/2412.15115>.
- Tony Ryan, Shu Wen Tay, P. D. Ryan, and C. Anthony Ryan. Systems 1 and 2 thinking processes and cognitive reflection testing in medical students. *Canadian Medical Education Journal*, 7:e97 – e103, 2016. URL <https://api.semanticscholar.org/CorpusID:11436274>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Yi Shen, Jian Zhang, Jieyun Huang, Shuming Shi, Wenjing Zhang, Jiangze Yan, Ning Wang, Kai Wang, and Shiguo Lian. Dast: Difficulty-adaptive slow-thinking for large reasoning models, 2025. URL <https://arxiv.org/abs/2503.04472>.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, EuroSys ’25, page 1279–1297. ACM, March 2025. doi: 10.1145/3689031.3696075. URL <http://dx.doi.org/10.1145/3689031.3696075>.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters, 2024. URL <https://arxiv.org/abs/2408.03314>.
- Zhaochen Su, Linjie Li, Mingyang Song, Yunzhuo Hao, Zhengyuan Yang, Jun Zhang, Guanjie Chen, Jiawei Gu, Juntao Li, Xiaoye Qu, et al. Openthinking: Learning to think with images via visual tool reinforcement learning. *arXiv preprint arXiv:2505.08617*, 2025.
- Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Hanjie Chen, and Xia Hu. Stop overthinking: A survey on efficient reasoning for large language models, 2025. URL <https://arxiv.org/abs/2503.16419>.
- Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, Chuning Tang, Congcong Wang, Dehao Zhang, Enming Yuan, Enzhe Lu, Fengxiang Tang, Flood Sung, Guangda Wei, Guokun Lai, Haiqing Guo, Han Zhu, Hao Ding, Hao Hu, Hao Yang, Hao Zhang, Haotian Yao, Haotian Zhao, Haoyu Lu, Haoze Li, Haozhen Yu, Hongcheng Gao, Huabin Zheng, Huan Yuan, Jia Chen, Jianhang Guo, Jianlin Su, Jianzhou Wang, Jie Zhao, Jin Zhang, Jingyuan Liu, Junjie Yan, Junyan Wu, Lidong Shi, Ling Ye,

- Longhui Yu, Mengnan Dong, Neo Zhang, Ningchen Ma, Qiwei Pan, Qucheng Gong, Shaowei Liu, Shengling Ma, Shupeng Wei, Sihan Cao, Siying Huang, Tao Jiang, Weihao Gao, Weimin Xiong, Weiran He, Weixiao Huang, Wenhao Wu, Wenyang He, Xianghui Wei, Xianqing Jia, Xingzhe Wu, Xinran Xu, Xinxing Zu, Xinyu Zhou, Xuehai Pan, Y. Charles, Yang Li, Yangyang Hu, Yangyang Liu, Yanru Chen, Yejie Wang, Yibo Liu, Yidao Qin, Yifeng Liu, Ying Yang, Yiping Bao, Yulun Du, Yuxin Wu, Yuzhi Wang, Zaida Zhou, Zhaoji Wang, Zhaowei Li, Zhen Zhu, Zheng Zhang, Zhexu Wang, Zhilin Yang, Zhiqi Huang, Zihao Huang, Ziyao Xu, and Zonghan Yang. Kimi k1.5: Scaling reinforcement learning with llms, 2025. URL <https://arxiv.org/abs/2501.12599>.
- Rui Wang, Hongru Wang, Boyang Xue, Jianhui Pang, Shudong Liu, Yi Chen, Jiahao Qiu, Derek Fai Wong, Heng Ji, and Kam-Fai Wong. Harnessing the reasoning economy: A survey of efficient reasoning for large language models. *arXiv preprint arXiv:2503.24377*, 2025.
- Heming Xia, Yongqi Li, Chak Tou Leong, Wenjie Wang, and Wenjie Li. Tokenskip: Controllable chain-of-thought compression in llms. *arXiv preprint arXiv:2502.12067*, 2025a.
- Heming Xia, Yongqi Li, Chak Tou Leong, Wenjie Wang, and Wenjie Li. Tokenskip: Controllable chain-of-thought compression in llms, 2025b. URL <https://arxiv.org/abs/2502.12067>.
- Silei Xu, Wenhao Xie, Lingxiao Zhao, and Pengcheng He. Chain of draft: Thinking faster by writing less, 2025. URL <https://arxiv.org/abs/2502.18600>.
- Edward Yeo, Yuxuan Tong, Morry Niu, Graham Neubig, and Xiang Yue. Demystifying long chain-of-thought reasoning in llms, 2025. URL <https://arxiv.org/abs/2502.03373>.
- Ping Yu, Jing Xu, Jason Weston, and Ilia Kulikov. Distilling system 2 into system 1, 2024. URL <https://arxiv.org/abs/2407.06023>.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. Dapo: An open-source llm reinforcement learning system at scale, 2025a. URL <https://arxiv.org/abs/2503.14476>.
- Zishun Yu, Tengyu Xu, Di Jin, Karthik Abinav Sankararaman, Yun He, Wenxuan Zhou, Zhouhao Zeng, Eryk Helenowski, Chen Zhu, Sinong Wang, et al. Think smarter not harder: Adaptive reasoning with inference aware optimization. *arXiv preprint arXiv:2501.17974*, 2025b.
- Qiyuan Zhang, Fuyuan Lyu, Zexu Sun, Lei Wang, Weixu Zhang, Wenyue Hua, Haolun Wu, Zhihan Guo, Yufei Wang, Niklas Muennighoff, Irwin King, Xue Liu, and Chen Ma. A survey on test-time scaling in large language models: What, how, where, and how well?, 2025. URL <https://arxiv.org/abs/2503.24235>.
- Yuze Zhao, Jintao Huang, Jinghan Hu, Xingjun Wang, Yunlin Mao, Daoze Zhang, Zeyinzi Jiang, Zhikai Wu, Baole Ai, Ang Wang, Wenmeng Zhou, and Yingda Chen. Swift: a scalable lightweight infrastructure for fine-tuning, 2024. URL <https://arxiv.org/abs/2408.05517>.